Moderné programovacie jazyky

Roman Rapco

Im – 29.4.2026

Bevy + Rust

Temu Jump King

- Jednoduchá hra napísaná v Ruste s pomocou Bevy engine.
- ECS = Entity Component System

Entity = objekt v hre

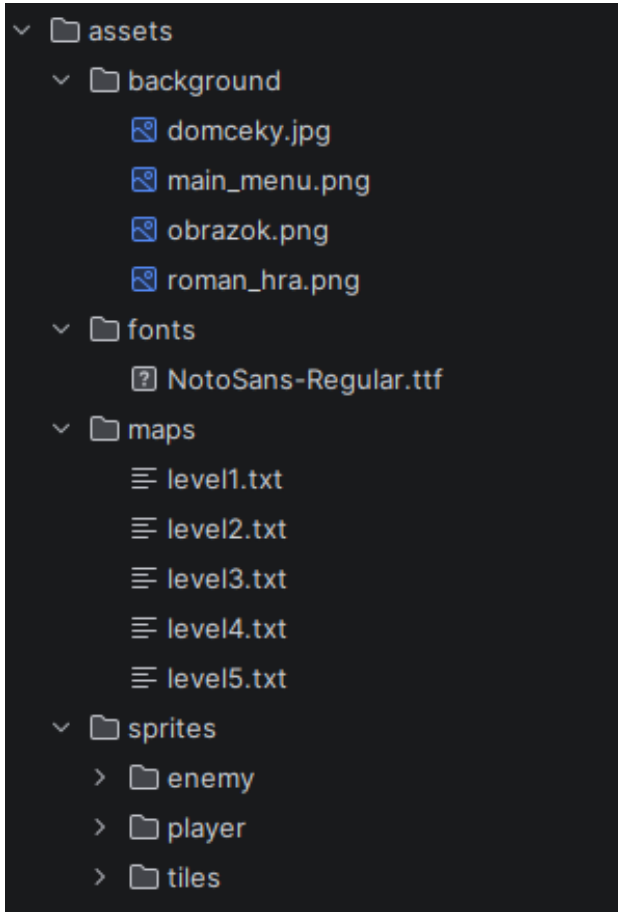
Component = data (pub struct)

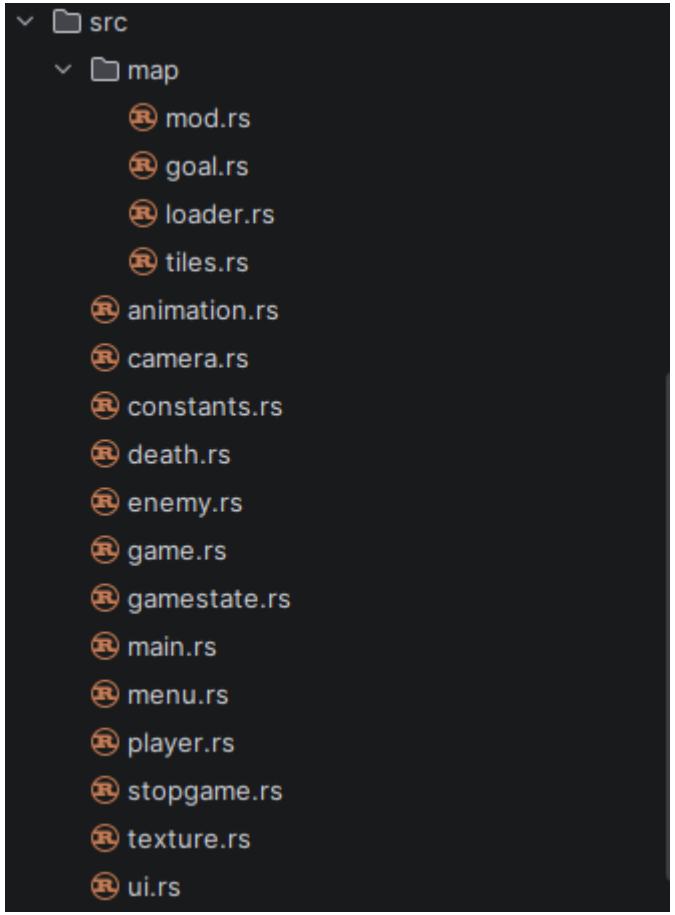
System = logika (funkcie)

- práca so stavmi

Architektúra projektu

- assets
- src
- core
- gameplay
- rendering/UI





Cargo.toml

```toml
[package]
name = "game"
version = "0.1.0"
edition = "2024"

[dependencies]
bevy = { version = "0.18.1" , features = ["exr"]}
avian2d = "0.6.1"
# bevy_dylib = "0.18.1"

# Enable a small amount of optimization in the dev profile.
[profile.dev]
opt-level = 1

# Enable a large amount of optimization in the dev profile for dependencies.
[profile.dev.package."*"]
opt-level = 3

# Enable more optimization in the release profile at the cost of compile time.
[profile.release]
# Compile the entire crate as one unit.
# Slows compile times, marginal improvements.
codegen-units = 1
# Do a second optimization pass over the entire program, including dependencies.
# Slows compile times, marginal improvements.
lto = "thin"

# Optimize for size in the wasm-release profile to reduce load times and bandwidth usage on web.
[profile.wasm-release]
# Default to release profile values.
inherits = "release"
# Optimize with size in mind (also try "z", sometimes it is better).
# Slightly slows compile times, great improvements to file size and runtime performance.
opt-level = "s"
# Strip all debugging information from the binary to slightly reduce file size.
strip = "debuginfo"
```

Main.rs

- inicializácia celého engine
- registrácia systémov
- nastavenie game state

```rust
use animation::{execute_animations, update_player_animation};
use camera::{camera_follow_player, setup_camera};
use constants::{WINDOW_HEIGHT, WINDOW_WIDTH, GRAVITY_PLAYER};
use enemy::update_enemies;
use death::{check_player_enemy_collision, death_input, death_countdown, check_death};
use gamestate::GameState;
use map::{load_map, check_goal_collision, victory_countdown, victory_input, cleanup_ingame};
use menu::{cleanup_menu, menu_action, setup_level_select, setup_menu, setup_leaderboard};
use player::{player_movement, spawn_player, debug_player_position};
use texture::{load_textures, setup_background};
use ui::{setup_ui, detect_height_change, update_height_ui, update_timer_ui, HeightChanged};
use stopgame::{freeze_entities};
use map::loader::{CurrentLevel, enter_ingame};
use game::{can_run_gameplay, return_to_menu};
```

```rust
Roman Rapco
mod animation;
Roman Rapco
mod camera;
Roman Rapco
mod constants;
Roman Rapco
mod enemy;
Roman Rapco
mod gamestate;
Roman Rapco
mod map;
Roman Rapco
mod menu;
Roman Rapco
mod player;
Roman Rapco
mod texture;
Roman Rapco
mod ui;
Roman Rapco
mod stopgame;
Roman Rapco
mod death;
Roman Rapco
mod game;
```

```rust
fn main() {
    App::new()
        .add_plugins(DefaultPlugins.set(WindowPlugin {
            primary_window: Some(Window {
                title: "Temu Jump King".into(),
                resolution: WindowResolution::new(WINDOW_WIDTH, WINDOW_HEIGHT),
                resizable: false,
                ..default()
            }),
            ..default()
        }))
        .insert_resource(ClearColor(Color::BLACK))
        .add_plugins(PhysicsPlugins::default())
        //.add_plugins(PhysicsDebugPlugin::default())
        .insert_resource(CurrentLevel { current: 1, max: 5 })
        .insert_resource(Gravity(Vec2::NEG_Y * GRAVITY_PLAYER))
        .init_state::<GameState>()
        .add_message::<HeightChanged>()
        .add_systems(Startup, setup_camera)

        .add_systems(OnEnter(GameState::Menu), setup_menu)
        .add_systems(Update, menu_action.run_if(in_state(GameState::Menu)))
        .add_systems(OnExit(GameState::Menu), cleanup_menu)

        .add_systems(OnEnter(GameState::LevelSelect), setup_level_select)
        .add_systems(Update, menu_action.run_if(in_state(GameState::LevelSelect)))
        .add_systems(OnExit(GameState::LevelSelect), cleanup_menu)

        .add_systems(OnEnter(GameState::Leaderboard), setup_leaderboard)
        .add_systems(Update, menu_action.run_if(in_state(GameState::Leaderboard)))
        .add_systems(OnExit(GameState::Leaderboard), cleanup_menu)
```

```rust
        .add_systems(
            OnEnter(GameState::LoadingLevel),
            (
                load_textures,
                setup_background,
                load_map,
                spawn_player,
                setup_ui,
                enter_ingame,
            ).chain(),
        )
        .add_systems(
            Update,
            (
                player_movement,
                //debug_player_position,
                update_player_animation,
                detect_height_change,
                update_height_ui,
                update_timer_ui,
                update_enemies,
                check_player_enemy_collision,
                check_goal_collision,
                camera_follow_player,
                check_death,
            )
                .chain()
                .run_if(in_state(GameState::InGame))
                .run_if(can_run_gameplay),
        )
```

```
.add_systems(
    Update,
    (
        execute_animations,
        victory_input,
        victory_countdown,
        death_input,
        death_countdown,
        freeze_entities,
        return_to_menu,
    )
        .run_if(in_state(GameState::InGame)),
)
.add_systems(OnExit(GameState::InGame), cleanup_ingame)
.run();
```

GameState

```rust
.init_state::<GameState>()
 .add_systems(OnEnter(GameState::Menu), setup_menu)
 .add_systems(Update, menu_action.run_if(in_state(GameState::Menu)))
 .add_systems(OnExit(GameState::Menu), cleanup_menu)
```

- state mašinka
- rôzne systémy bežia podľa stavu

```rust
Roman Rapco
#[derive(States, Debug, Clone, Copy, Eq, PartialEq, Hash, Default)]
pub enum GameState {
    #[default]
    Menu,
    LevelSelect,
    LoadingLevel,
    Leaderboard,
    InGame,
}

Roman Rapco
#[derive(Component)]
pub struct InGameEntity;
```

(Main) Menu

```
5 usages  Roman Rapco
#[derive(Component)]
pub struct MainMenuUI;

15 usages  Roman Rapco
#[derive(Component)]
pub enum MenuButtonAction {
    Play,
    LevelSelect,
    Leaderboard,
    SelectLevel(usize),
    Back,
    Quit,
}
```

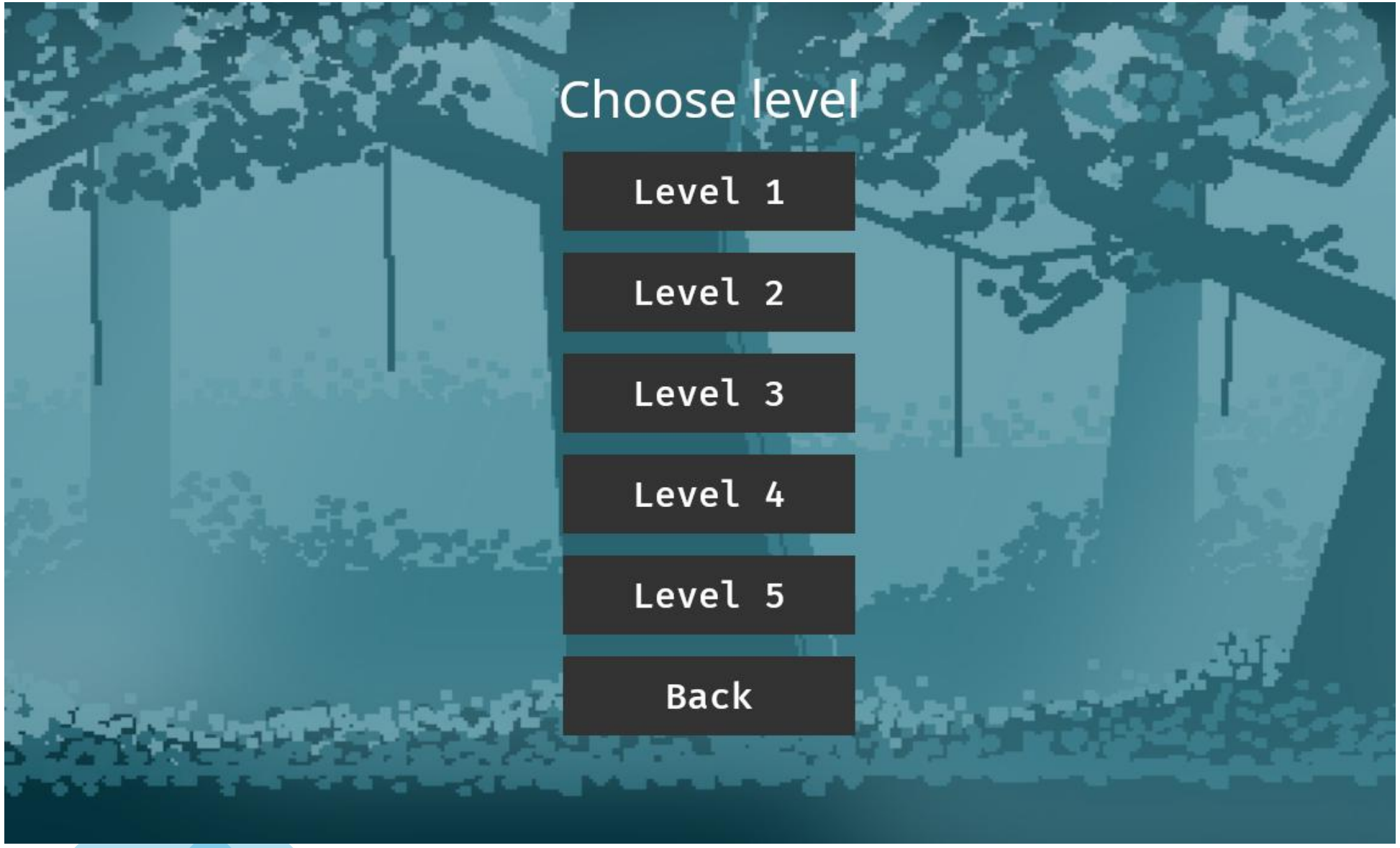
```
pub fn setup_menu(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
) {
    let font = asset_server.load("fonts/NotoSans-Regular.ttf");
    let background = asset_server.load("background/main_menu.png");

    spawn_background(&mut commands, background);
```

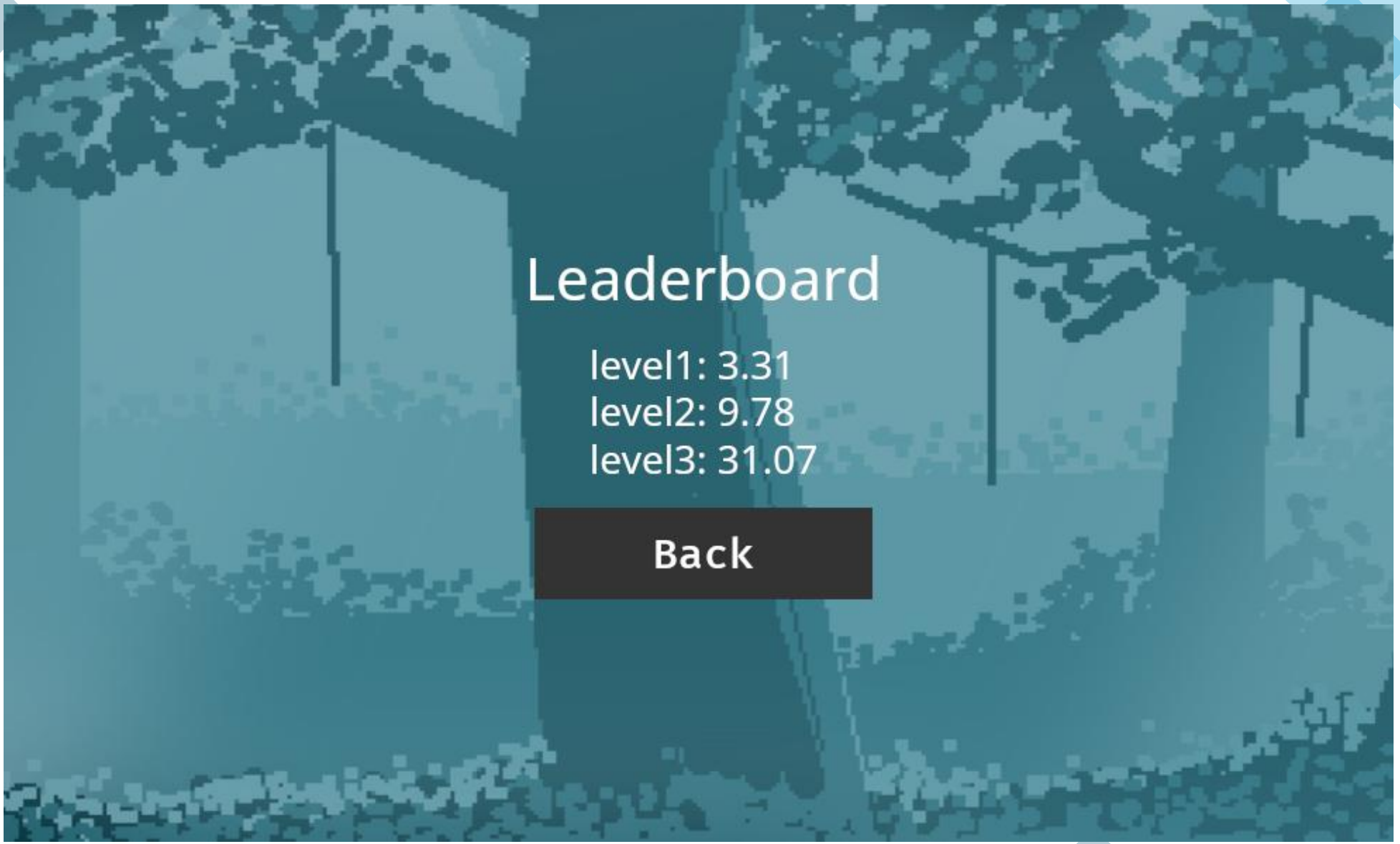
```
spawn_button(parent, text: "Play", MenuButtonAction::Play);
spawn_button(parent, text: "Level Change", MenuButtonAction::LevelSelect);
spawn_button(parent, text: "Leaderboard", MenuButtonAction::Leaderboard);
spawn_button(parent, text: "Quit", MenuButtonAction::Quit);
```

```
for level in 1..=current_level.max {
    let label = format!("Level {}", level);
    spawn_button(parent, &label, MenuButtonAction::SelectLevel(level));
}

spawn_button(parent, text: "Back", MenuButtonAction::Back);
```

```
match action {
    MenuButtonAction::Play => {
        current_level.current = 1;
        next_state.set(GameState::LoadingLevel);
    }
    MenuButtonAction::LevelSelect => {
        next_state.set(GameState::LevelSelect);
    }
    MenuButtonAction::Leaderboard => {
        next_state.set(GameState::Leaderboard);
    }
    MenuButtonAction::SelectLevel(level : usize) => {
        current_level.current = *level;
        next_state.set(GameState::LoadingLevel);
    }
    MenuButtonAction::Back => {
        next_state.set(GameState::Menu);
    }
    MenuButtonAction::Quit => {
        exit.write(AppExit::Success);
    }
}
```

Kráľ skoku
Play
Level Change
Leaderboard
Quit


Choose level
Level 1
Level 2
Level 3
Level 4
Level 5
Back


Leaderboard
level1: 3.31
level2: 9.78
level3: 31.07
Back

Player

```
3 usages   Roman Rapco
#[derive(Component)]
pub struct Player;

1 usage   Roman Rapco
#[derive(Component)]
pub struct PlayerSprite;
```

- player_movement, spawn_player
- pohyb hráča
- gravitácia
- input handling

```
pub fn spawn_player(
    mut commands: Commands,
    spawn_point: Res<PlayerSpawnPoint>,
    textures: Res<GameTextures>,
) {
    commands
        .spawn((
            Transform::from_xyz(spawn_point.0.x, spawn_point.0.y, 1.0),
            Visibility::Visible,
            Player,
            InGameEntity,
            RigidBody::Dynamic,
            Collider::rectangle(PLAYER_WIDTH, PLAYER_HEIGHT),
            LinearVelocity::ZERO,
            LockedAxes::ROTATION_LOCKED,
            Friction::ZERO,
            Restitution::ZERO,
        ))
        .with_children(|parent| {
            parent.spawn((
                Sprite {
                    image: textures.player_idle.clone(),
                    texture_atlas: Some(TextureAtlas {
                        layout: textures.player_idle_layout.clone(),
                        index: 0,
                    }),
                    ..default()
                },
                Transform::from_xyz(0.0, 15.0, 0.0),
                PlayerSprite,
                PlayerAnimation::Idle,
                AnimationConfig::new(0, 6, 8),
            ));
```

```rust
// movement
velocity.x = 0.0;
if keyboard_input.pressed(KeyCode::KeyA) {
    velocity.x -= PLAYER_SPEED;
}
if keyboard_input.pressed(KeyCode::KeyD) {
    velocity.x += PLAYER_SPEED;
}
```

```rust
// ground check
let on_ground :bool =
    spatial_query
        .cast_ray(center, Dir2::NEG_Y, ray_distance, true, &filter)
        .is_some()
        || spatial_query
        .cast_ray(left, Dir2::NEG_Y, ray_distance, true, &filter)
        .is_some()
        || spatial_query
        .cast_ray(right, Dir2::NEG_Y, ray_distance, true, &filter)
        .is_some();

// skok
if keyboard_input.just_pressed(KeyCode::Space) && on_ground {
    velocity.y = JUMP_FORCE;
}

if keyboard_input.just_pressed(KeyCode::KeyW) && on_ground {
    velocity.y = JUMP_FORCE;
}
```

Enemy

spawn_enemy
update_enemies

```rust
match enemy_type {
    EnemyType::Walking => {
        entity.insert(WalkingState {
            direction: if start_left { -1.0 } else { 1.0 },
            distance_moved: 0.0,
            max_distance: TILE_SIZE_X * 4.0,
        });
    }
    EnemyType::Jumping => {
        entity.insert(JumpingState {
            jump_timer: Timer::from_seconds(2.0, TimerMode::Repeating),
        });
    }
    _ => {}
}
```

```rust
2 usages  Roman Rapco
pub const ENEMY_WIDTH: f32 = 20.0;
3 usages  Roman Rapco
pub const ENEMY_HEIGHT: f32 = 30.0;

10 usages  Roman Rapco
#[derive(Component, Clone, PartialEq)]
pub enum EnemyType {
    Standing,
    Walking,
    Jumping,
}

2 usages  Roman Rapco
#[derive(Component)]
pub struct Enemy;

2 usages  Roman Rapco
#[derive(Component)]
pub struct WalkingState {
    pub direction: f32,
    pub distance_moved: f32,
    pub max_distance: f32,
}

2 usages  Roman Rapco
#[derive(Component)]
pub struct JumpingState {
    pub jump_timer: Timer,
}
```

```rust
match enemy_type {
    EnemyType::Standing => {
        velocity.x = 0.0;
    }
    EnemyType::Walking => {
        if let Some(mut state) = walking {
            velocity.x = state.direction * ENEMY_SPEED;
            state.distance_moved += ENEMY_SPEED * time.delta_secs();

            if state.distance_moved >= state.max_distance {
                state.direction *= -1.0;
                state.distance_moved = 0.0;
            }
        }
    }
    EnemyType::Jumping => {
        velocity.x = 0.0;
        if let Some(mut state) = jumping {
            state.jump_timer.tick(time.delta());
            if state.jump_timer.just_finished() {
                velocity.y = 400.0;
            }
        }
    }
}
```

Animation

- update_player_animation
- execute_animations

```rust
if velocity.x < -0.1 {
    sprite.flip_x = false;
} else if velocity.x > 0.1 {
    sprite.flip_x = true;
}

let new_anim :PlayerAnimation = if velocity.x.abs() > 0.1 {
    PlayerAnimation::Run
} else {
    PlayerAnimation::Idle
};
```

```rust
impl AnimationConfig {
    Roman Rapco
    pub fn new(first: usize, last: usize, fps: u8) -> Self {
        Self {
            first_sprite_index: first,
            last_sprite_index: last,
            fps,
            frame_timer: Self::timer_from_fps(fps),
        }
    }

    Roman Rapco
    pub fn timer_from_fps(fps: u8) -> Timer {
        Timer::new(
            Duration::from_secs_f32(1.0 / fps as f32),
            TimerMode::Repeating,
        )
    }
}
```

```rust
#[derive(Component, Clone, PartialEq)]
pub enum PlayerAnimation {
    Idle,
    Run,
}

6 usages  Roman Rapco
#[derive(Component)]
pub struct AnimationConfig {
    pub first_sprite_index: usize,
    pub last_sprite_index: usize,
    pub fps: u8,
    pub frame_timer: Timer,
}
```

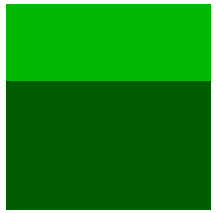
```rust
match new_anim {
    PlayerAnimation::Idle => {
        sprite.image = textures.player_idle.clone();
        sprite.texture_atlas = Some(TextureAtlas {
            layout: textures.player_idle_layout.clone(),
            index: 0,
        });
        *config = AnimationConfig::new(0, 6, 8);
    }
    PlayerAnimation::Run => {
        sprite.image = textures.player_run.clone();
        sprite.texture_atlas = Some(TextureAtlas {
            layout: textures.player_run_layout.clone(),
            index: 0,
        });
        *config = AnimationConfig::new(0, 7, 10);
    }
}
```

Texture

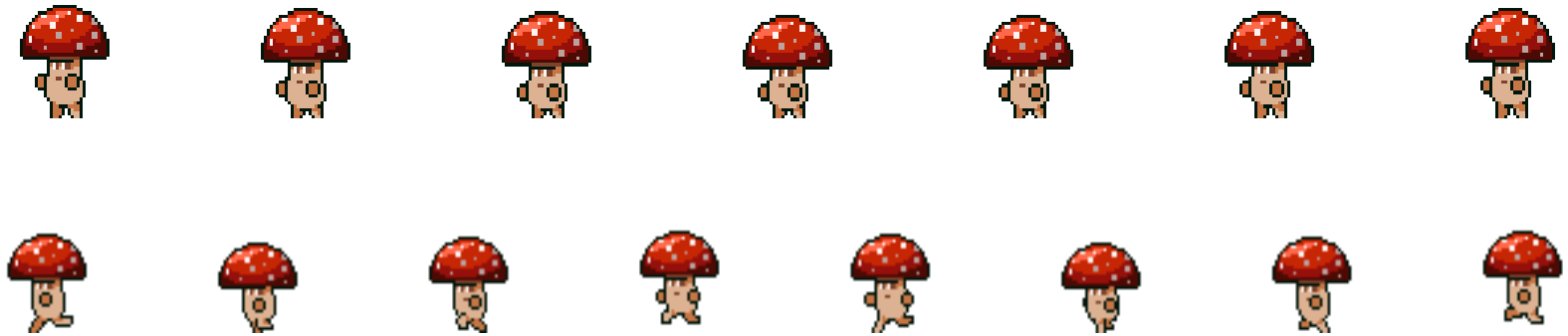
- setup_background
- load_textures

```rust
#[derive(Resource)]
pub struct GameTextures {
    pub player_idle: Handle<Image>,
    pub player_run: Handle<Image>,
    pub player_death: Handle<Image>,

    pub player_idle_layout: Handle<TextureAtlasLayout>,
    pub player_run_layout: Handle<TextureAtlasLayout>,
    pub player_death_layout: Handle<TextureAtlasLayout>,

    pub platform: Handle<Image>,
}
```

```rust
pub fn load_textures(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut texture_atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
) {
    let player_idle_layout = texture_atlas_layouts.add(
        TextureAtlasLayout::from_grid(UVec2::new(80, 64), 7, 1, None, None)
    );
    let player_run_layout = texture_atlas_layouts.add(
        TextureAtlasLayout::from_grid(UVec2::new(80, 64), 8, 1, None, None)
    );
    let player_death_layout = texture_atlas_layouts.add(
        TextureAtlasLayout::from_grid(UVec2::new(64, 64), 15, 1, None, None)
    );

    commands.insert_resource(GameTextures {
        player_idle: asset_server.load("sprites/player/Mushroom-Idle.png"),
        player_run: asset_server.load("sprites/player/Mushroom-Run.png"),
        player_death: asset_server.load("sprites/player/Mushroom-Die.png"),
        player_death_layout,
        player_idle_layout,
        player_run_layout,
        platform: asset_server.load("sprites/tiles/platform.png"),
    });
```

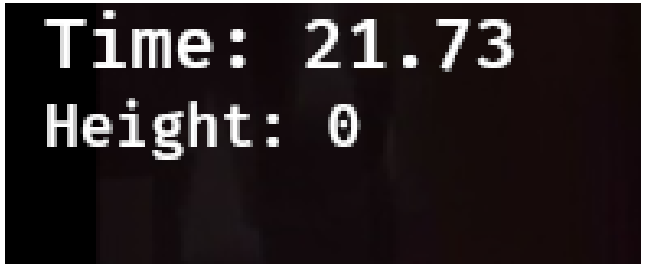


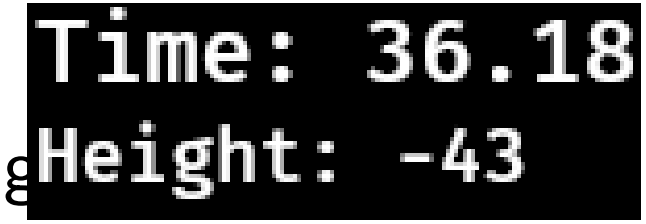
https://www.piskelapp.com/



UI in game

- setup_ui
- update_timer_ui
- detect_height_chang
- update_height_ui
- Event = message (new)






```rust
pub fn update_timer_ui(
    time: Res<Time>,
    mut level_timer: ResMut<LevelTimer>,
    mut text_query: Query<&mut Text, With<TimerText>>,
) {
    level_timer.seconds += time.delta_secs();

    let Ok(mut text) = text_query.single_mut() else {
        return;
    };

    **text = format!("Time: {:.2}", level_timer.seconds);
}
```

```rust
#[derive(Message, Clone)]
pub struct HeightChanged(pub f32);

2 usages  Roman Rapco
#[derive(Component)]
pub struct HeightText;

2 usages  Roman Rapco
#[derive(Component)]
pub struct TimerText;

2 usages  Roman Rapco
#[derive(Resource)]
pub struct LevelTimer {
    pub seconds: f32,
}
```

```rust
pub fn detect_height_change(
    player_query: Query<&Transform, With<Player>>,
    mut last_height: Local<f32>,
    mut writer: MessageWriter<HeightChanged>,
) {
    let Ok(transform) = player_query.single() else {
        return;
    };

    let current_height = (((transform.translation.y) / 10.0) - 10.0).round();

    if (current_height - *last_height).abs() >= 1.0 {
        *last_height = current_height;
        writer.write(HeightChanged(current_height));
    }
}
```

Death / stopgame

- spawn_death_ui
- check_player_enemy_collision
- death_input = space after death
- death_countdown = 60 sekúnd
- check_death = fall death

- freeze_entities
- save_level_time

```rust
pub fn check_death(
    mut commands: Commands,
    player: Single<&Transform, With<Player>>,
    dead: Option<Res<DeathTimer>>,
) {
    if player.translation.y < -400.0 {
        if dead.is_none() {
            println!("You died");
            spawn_death_ui(&mut commands);
            commands.insert_resource(DeathTimer(Timer::from_seconds(60.0, TimerMode::Once)));
        }
        return;
    }
}
```

```rust
let player_pos = player_transform.translation.truncate();

for enemy_transform in &enemy_query {
    let enemy_pos = enemy_transform.translation.truncate();

    let padding :f64 = 1.0;

    let overlap_x :bool =
        (player_pos.x - enemy_pos.x).abs() <= (PLAYER_WIDTH / 2.0 + ENEMY_WIDTH / 2.0 + padding);
    let overlap_y :bool =
        (player_pos.y - enemy_pos.y).abs() <= (PLAYER_HEIGHT / 2.0 + ENEMY_HEIGHT / 2.0 + padding);

    if overlap_x && overlap_y {
        println!("You died");
        spawn_death_ui(&mut commands);
        commands.insert_resource(DeathTimer(Timer::from_seconds(60.0, TimerMode::Once)));
        break;
    }
}
```

```rust
pub fn death_countdown(
    time: Res<Time>,
    dead: Option<ResMut<DeathTimer>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    let Some(mut dead) = dead else {
        return;
    };

    dead.0.tick(time.delta());

    if dead.0.just_finished() {
        next_state.set(GameState::LoadingLevel);
    }
}
```

```rust
pub fn death_input(
    keyboard_input: Res<ButtonInput<KeyCode>>,
    mut next_state: ResMut<NextState<GameState>>,
    dead: Option<Res<DeathTimer>>,
) {
    if dead.is_none() {
        return;
    }

    if keyboard_input.just_pressed(KeyCode::Space) {
        next_state.set(GameState::LoadingLevel);
    }
}
```

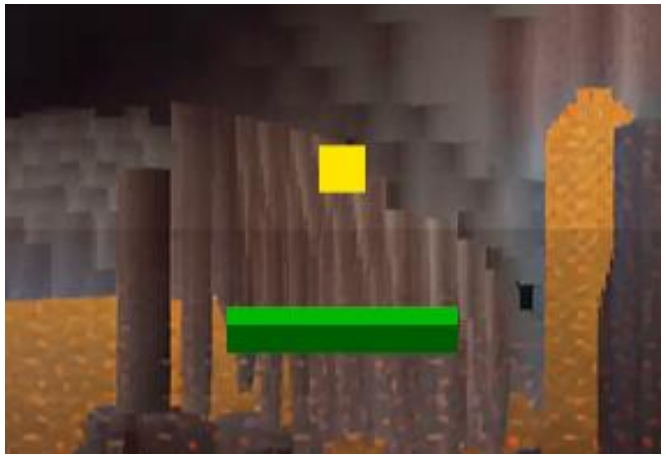
```rust
pub fn save_level_time(level: usize, time: f32) {
    let path :&str = "times.txt";

    let content = read_to_string(path).unwrap_or_default();
    let mut lines: Vec<String> = content.lines().map(|line| line.to_string()).collect();

    let level_prefix = format!("level{}:", level);
    let new_line = format!("level{}: {:.2}", level, time);

    let mut found :bool = false;

    for line in &mut lines {
        if line.starts_with(&level_prefix) {
            found = true;

            let old_time_text = line
                .split(':')
                .nth(1)
                .unwrap_or("")
                .trim();

            let old_time = old_time_text.parse::<f32>().unwrap_or(f32::MAX);
```

```rust
pub fn freeze_entities(
    mut query: Query<&mut LinearVelocity, Or<(With<Player>, With<Enemy>)>>,
    dead: Option<Res<DeathTimer>>,
    victory: Option<Res<VictoryTimer>>,
) {
    if dead.is_none() && victory.is_none() {
        return;
    }

    for mut velocity in &mut query {
        velocity.x = 0.0;
        velocity.y = 0.0;
    }
}
```

```rust
            if time < old_time {
                *line = new_line.clone();
            }

            break;
        }
    }

    if !found {
        lines.push(new_line);
    }

    let final_text = lines.join("\n");

    let mut file = OpenOptions::new()
        .create(true)
        .write(true)
        .truncate(true)
        .open(path)
        .unwrap();

    file.write_all(final_text.as_bytes()).unwrap();
```

Goal

- spawn_goal
- spawn_victory_ui
- check_goal_collision
- victory_input = space + next level
- victory_countdown = odpocet + next level
- mod = cleanup_ingame



```
.insert_resource(CurrentLevel { current: 1, max: 5 })
```

Loader / tiles

- enter_ingame
- load_map
- c, m, w, j, s, p

```rust
pub fn load_map(
    mut commands: Commands,
    textures: Res<GameTextures>,
    current_level: Res<CurrentLevel>,
) {
    let path = format!("assets/maps/level{}.txt", current_level.current);

    let content = match fs::read_to_string(&path) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("Chyba pri nacitani mapy {}: {}", path, e);
            return;
        }
    };

    let lines: Vec<&str> = content.lines().collect();
    let map_height = lines.len();
```

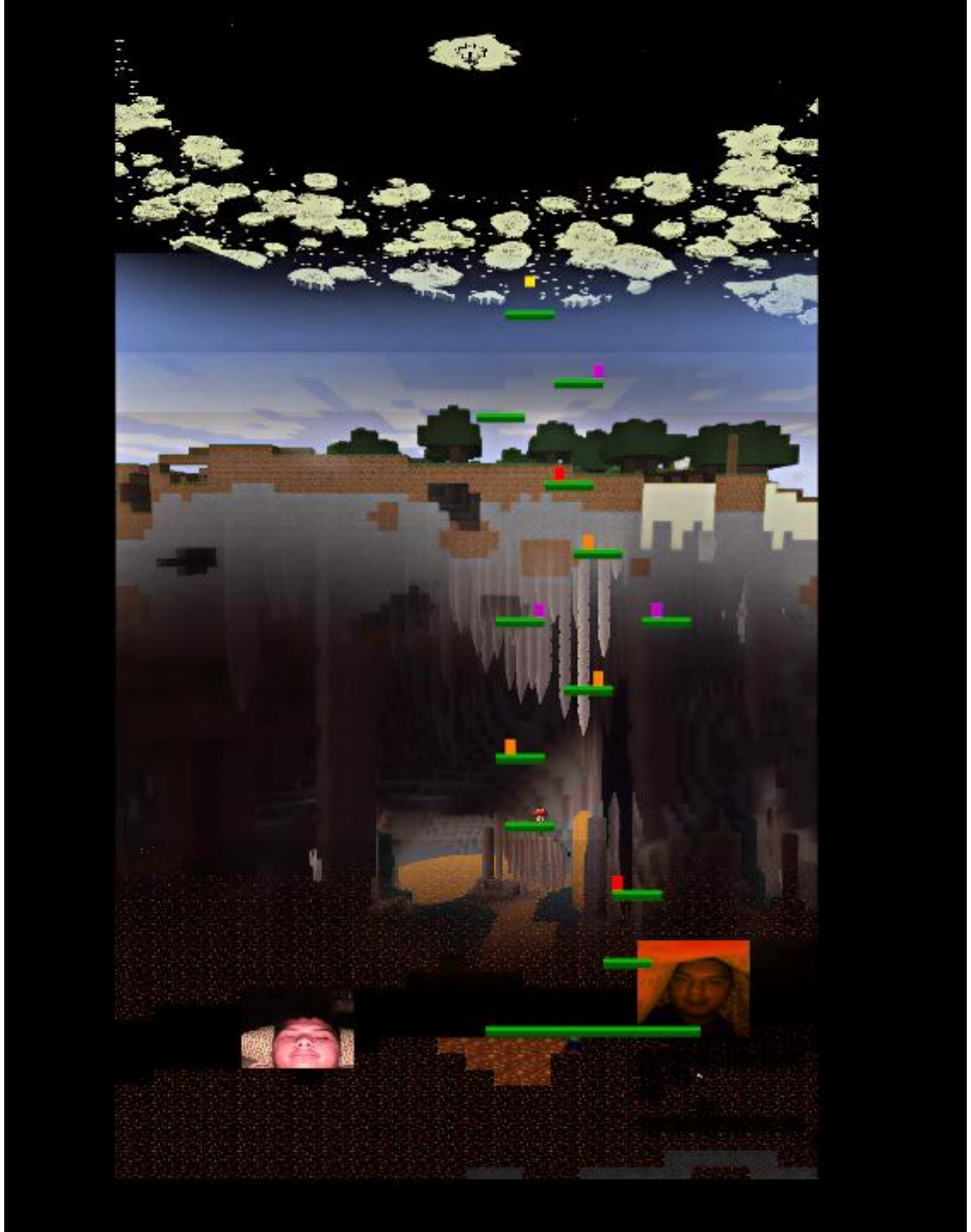
```rust
for (row, line) in lines.iter().enumerate() {
    let mut col :i32 = 0;
    let chars: Vec<char> = line.chars().collect();

    while col < chars.len() {
        let ch = chars[col];

        let x = col as f32 * TILE_SIZE_X + TILE_SIZE_X / 2.0;
        let y = (map_height - 1 - row) as f32 * (TILE_SIZE_Y + ROW_GAP) + TILE_SIZE_Y / 2.0;

        match ch {
            '_' => {
                let start_col :i32 = col;
                while col < chars.len() && chars[col] == '_' {
                    col += 1;
                }
                let count = col - start_col;
                let width = count as f32 * TILE_SIZE_X;
                let cx = start_col as f32 * TILE_SIZE_X + width / 2.0;

                spawn_tile(
                    &mut commands,
                    &textures,
                    cx,
                    y,
                    width,
                    TILE_SIZE_Y,
                );
                continue;
            }
```

```rust
pub fn enter_ingame(mut next_state: ResMut<NextState<GameState>>) {
    next_state.set(GameState::InGame);
}
```

```rust
pub fn spawn_tile(
    commands: &mut Commands,
    textures: &Res<GameTextures>,
    x: f32, y: f32,
    width: f32, height: f32,
) {
    commands.spawn((
        Sprite {
            image: textures.platform.clone(),
            custom_size: Some(Vec2::new(width, height)),
            ..default()
        },
        Transform::from_xyz(x, y, 0.0),
        InGameEntity,
        RigidBody::Static,
        Collider::rectangle(width, height),
    ));
}
```

```rust
'/' => {
    spawn_slope(&mut commands, x, y, false);
}
'\\' => {
    spawn_slope(&mut commands, x, y, true);
}
'p' => {
    spawn_point = Vec2::new(x, y);
}
'c' => {
    spawn_goal(&mut commands, x, y);
}
's' => {
    spawn_enemy(&mut commands, x, y, EnemyType::Standing, false);
}
'w' => {
    spawn_enemy(&mut commands, x, y, EnemyType::Walking, false);
}
'm' => {
    spawn_enemy(&mut commands, x, y, EnemyType::Walking, true);
}
'j' => {
    spawn_enemy(&mut commands, x, y, EnemyType::Jumping, false);
}
_ => {}
```

```
#########################
#     c                 #
#   _____               #
#           j           #
#        _____          #
#_____                  #
#        s              #
#       _____           #
#              m        #
#          _____        #
#      j           j    #
#  _____          _____ #
#         w             #
#         _____         #
#      m                #
#  _____                #
#                       #
#   _____               #
#              s        #
#              _____    #
#                       #
#             _____     #
#            p          #
# _____________________#
#########################
```



```
##########################
#     c                  #
#   _____                #
#           j            #
#        _____           #
#_____                   #
#           p            #
# _______________________#
##########################
```

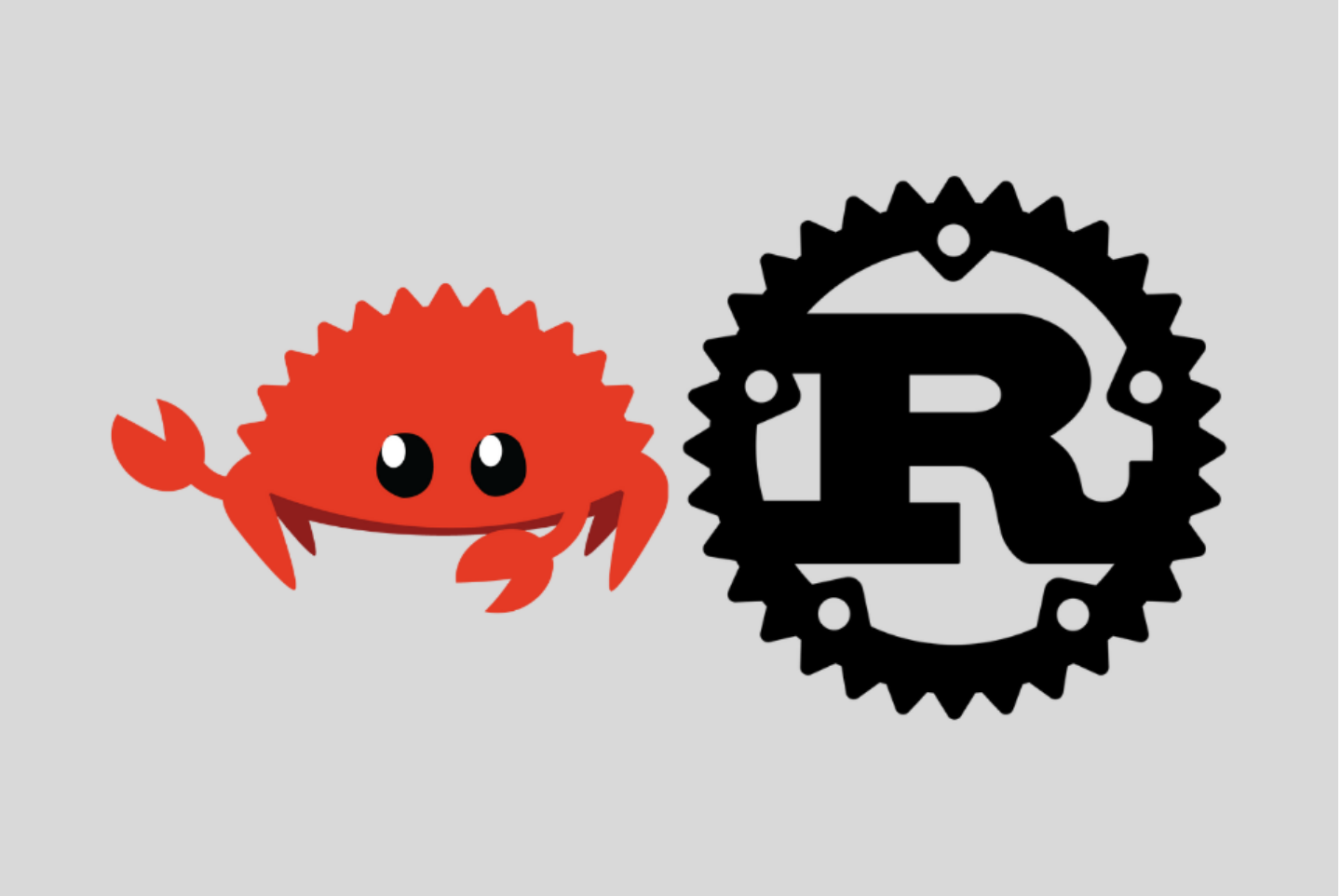
```
##################################################
#          c                                     #
#        _____                m          __      #
#              __        _____                   #
#                                                #
#                                 __             #
#        __                      j        j      #
#                               ___       __     #
#                     ___                        #
#              _                                 #
#          j            j                        #
#         __            ___                      #
#     m                                          #
#  _____     s                                   #
#           ___                    __            #
#                  __                            #
#                        w                       #
#                       _____                    #
#                                     j          #
#                            w        ___        #
#                      m      ________           #
#            w       ___                         #
#           _____                                #
#           p                                    #
# _______________________                        #
##################################################
```

```
#     c                  #
#   _____                #
#           j            #
#w        _____          #
#_____                   #
#        s  j            #
#       _____            #
#          w             #
#        _____           #
#     j           j      #
# _____         _____   #
#            m           #
#        _____           #
# w                      #
# _____                  #
#      m                 #
#   _____                #
#             w          #
#            _____       #
#                m       #
#           _____        #
#      w                 #
#      _____             #
#             m          #
#          _____         #
#     j           j      #
# _____         _____   #
#            m           #
#        _____           #
# w                      #
# _____                  #
#      m                 #
#   _____                #
#                m       #
#            _____       #
#           w            #
#            _____       #
#          p             #
# _______________________#
```

Temu Jump King

Rust koncepty

- Ownership & borrowing
- Enums
- Moduls
- Match
- Query
- Option
- Structs
- Resources



Diskusia

- viac levelov
- death animácia
- sprites for enemies
- double jump ?
- ...

Ďakujem za pozornosť